Credit Agreement Extraction

End to end, and auditable at every step

138 fields · 5 documents · 690 results

Every value cited, checked and scored

A credit agreement does not give up its facts facts easily

400k–1.1M

characters in a single agreement

138

fields a CLO desk needs out of it

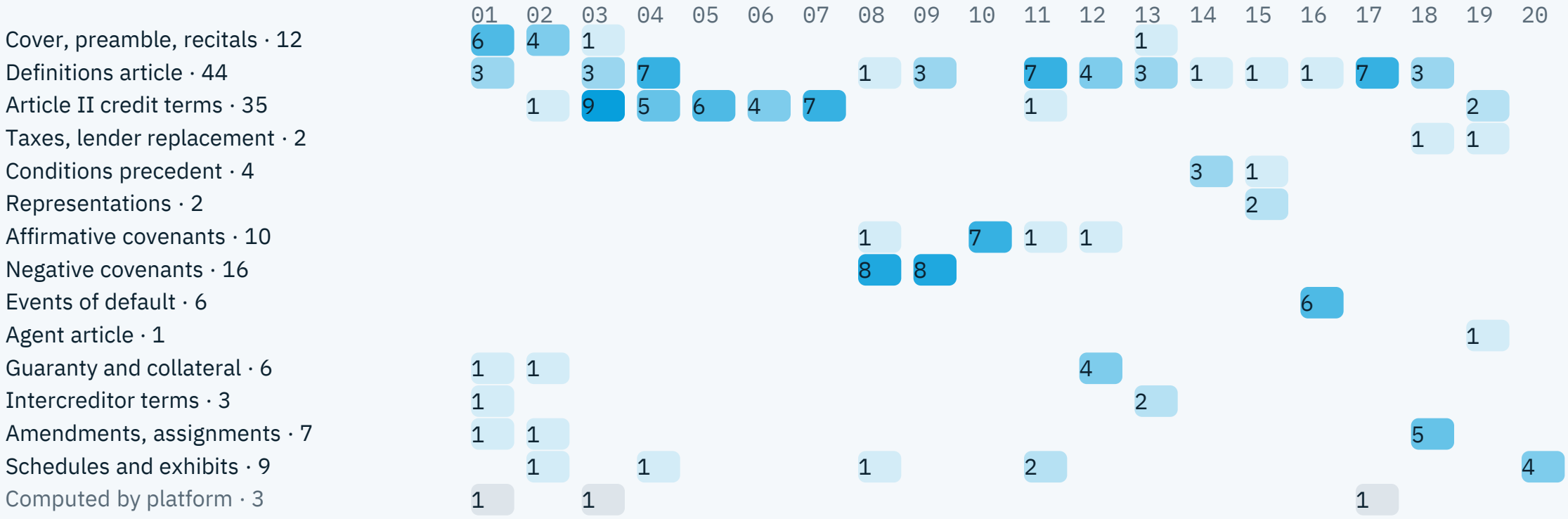
3

things every one of those fields must carry: a citation, a check, a confidence

Today the desk gets there by reading. That is analyst hours, per document, every time one lands.

Which section feeds which item

Items CA-01 to CA-20 across · cell = fields read from that section



The definitions article feeds 13 of the 20 items. Five items read from a single section.

What was built: one pipeline, seven stages

Deterministic wherever it can be, a model call only where judgment is needed, and a trace of every step.



Prepare	Locate	Extract	Align	Validate	Verify	Route
Clean text, section index, defined-term glossary	Hand-written plan per field picks the passages to read	One structured call per group of fields, schema enforced	Every quote pinned to exact characters, or the record is flagged	138 rules and cross-checks; dates resolved; formats enforced	Second opinion only where a check disagreed	Tier, confidence band, reviewer queue, deliverables
no model	no model	**model**	no model	no model	**model**	no model

Two of the seven stages call the model. The other five are code that runs the same way every time.

The model never decides where to look and never chooses a value on its own: it reads the passages the engine hands it and answers within a fixed schema.

Every value it returns must be found verbatim in the document; if it cannot be, the record keeps its value but loses its citation and goes to a reviewer.

Prepare: what the engine knows before a call

Deterministic and free. One command turns a raw filing into six files that every later stage reads; document 01 takes under a second.

1 Normalise — `normalise.py`

One canonical text with stable character offsets. Tables become rows; a PDF's running heads are dropped. Every citation is an offset into it.

2 Segment — `segment.py`

The contents page is found and each entry placed on its heading in the body. Schedules and exhibits are scanned from the back.

3 Definitions — `definitions.py`

Chosen by title and size; every definition head is parsed and a graph links each term to the terms it uses, cycles included.

4 References — `references.py`

Every "Section 2.04(b)" or "Schedule 6.01" is resolved against the index, and each section is linked to the terms it uses.

5 Composite probe — `composite.py`

Does the filing open as an amendment with the agreement attached behind it? If so the attached cover becomes the origin.

6 Pseudo-sections — `prepared.py`

Passages the index cannot name are named: the cover, preamble, recitals, the amendment wrapper, signatures, and the tail.

Document 01, as prepared

File	What it holds for document 01
`canonical.txt`	502,275 characters, 1,459 table rows
`sections.json`	161 sections: 135 placed, 26 schedules
`definitions.json`	226 terms, 633 links, 29 cycles
`references.json`	520 pointers, 432 resolved; 1,362 term uses
`meta.json`, `trace.jsonl`	the hash and counts; 148 trace events

Why it is a stage of its own

Free to re-run. No model, so a better parser reaches every document in seconds and changes no hash.

Every later stage reads these files, not the filing — locate, the resolver, the follower, the tool.

Traced. Every decision is an event the trace page shows under stage 0.

Inside the locate stage

A PLAN, NOT A SEARCH

Every field carries a location description and a rule. No embeddings, no chunk ranking.

Anchors

Fixed parts, taken by position

Sections

Section titles and ids

Defined terms

The agreement's own dictionary: exact, suffix, contains, then fuzzy. Dependencies come too.

Merge and cap

Spans within 400 characters join. A region is capped at 120,000 characters, a field at 200,000.

Regions, not documents

Only the located regions are sent, each one uncut and labelled. One structured call per field.

Citation by alignment

The model quotes. The aligner finds the quote and sets the offsets.

The prompt for one item: CA-02 on document 01

Four layers, one call, six fields. Each field's block is rendered from its dictionary row; only the rules and four reminders are hand-written.

1 System prompt · extract_system_2026-09-21.txt

Ten rules in order of importance, the same for every item and document:

- Only what the text supports
- Never infer; wording in stated_as
- No quote, no value — enums too
- Quotes copied character by character
- Quote the passage that states it
- 1:many: one entry per real instance
- Enums from the allowed list
- ISO dates; numbers as numbers
- A supplied definition wins
- Object parts go in key_quotes

2 Document context, cached

The 12 passages the locator chose for the six fields — 78,858 characters, each under a header such as "===== REGION Section 10.09 | canonical offset 353716–356314 =====".

3 Instructions · extract_instructions_2026-09-05b.txt

"Extract item CA-02 from the regions above, for document 01." Then six field blocks, like this one:

```
- CA-02-02 `party_role` (Enum, 1:many)
  definition: Role in the preamble
  format: Controlled list
  allowed values: the fourteen roles
```

Then four reminders: return every field; no quote, no value; work 1:many instance by instance; fill only the fields listed.

4 Schema, as a constraint

Sent as the structured-output format, not as prose: one object, six keys, 9,839 bytes. The fourteen roles are a closed list, and every value carries quote and stated_as slots. Enforced during generation, re-checked in code.

What came back, document 01

One reply for all six fields: 1,031 fresh and 34,024 cached input tokens, 4,079 output tokens

27 records split out of it: 6 parties, 8 roles, 4 commitment rows, each quote aligned.

Stage 2 of the trace page shows the block as rendered; stage 3 shows the call.

Where the 690 landed

Every one of the 138 fields, in each of the five documents, banded.



423 · 61.3%

Green — extracted, validated, cross-checked and agreed

41 · 5.9%

Amber — found, with a caveat worth a reviewer's eye

22 · 3.2%

Red — routed to a human, deliberately not guessed

204 · 29.6%

Empty — the filing does not contain it; recorded as absent

53 fields

carry a value on all five documents — the dependable core.

12 of 22

reds come from the same five fields — covenant metric components, structural protections, covenant category, borrowing base component and the dependency graph.

12 fields

are absent from all five — ECF sweep, PIK toggle, ticking fee, standstill period. That is 60 of the 204 empties; the other 144 are fields some deals have and others do not.

Seven fields across five agreements

The same schema over a 2010 revolver, a second lien, an ABL, an amendment with its agreement attached, and a REIT facility.

	Doc 01 Revolver, 2010	**Doc 02** Second Lien, 2020	**Doc 04** Amendment, 2023	**Doc 05** ABL, PQ Corp, 2016	**Doc 06** REIT facility, 2019
Closing date CA-01-04	2010-09-09, from “the date hereof”	2019-02-01, in the definition	2015-07-31, attached agreement	Not a date: when Section 4.01 is met	Not a date: when conditions are met
Currencies CA-01-10	USD, from “Dollar”	USD, same definition	USD, from “Dollars” and “$”	USD, CAD, GBP, EUR	USD, from “U.S. Dollars”
Maturity date CA-03-09	2013-09-09, in the definition	2027-02-01	2028-01-17 and 2026-12-17	2021-05-04, from “five years after Closing”	2023-11-26, via the fallback
Lender commitments CA-02-05	4 lenders; BofA 37.5%	Not stated — schedule not filed	1 row: $180m revolver	45 rows, 10 lenders; sums to $200m	3 lenders; Bank of Montreal $50m
Margin grid CA-04-03	3 tiers: LIBOR 200 / 175 / 150	Not stated — fixed 850 bps	Not stated as a grid	3 categories: LIBO 150 / 175 / 200	4 levels: base 35 to 95 bps
Covenant levels CA-08-04	Leverage 1.50x; FCCR 1.35x then 1.50x	None located	FCCR 1.00x after closing	FCCR 1.00 : 1.00	5 covenants: leverage 0.60x, FCCR 1.50x
Required lenders CA-18-04	66.7%, or 50.01% with four or more	50.01%	50.01%, three lender classes	50.01%; Super Majority 66.67%	66.67%

Green: cited, and every engine check passed — 29 of the 35 cells. Grey: the document does not state the field, and the record says why — 6 cells. No cell was hand edited.

Deep dive: a maturity date the document never prints

CA-03-09 on document 05. The engine walks the defined terms, resolves the date, and records every step and the one assumption.

The field asks for

Maturity date of the Initial Revolving Loans

The commitment definition points at a defined term, not a date: “Initial Revolving Credit Maturity Date”.

→

That term says

“the date that is five years after the Closing Date”

Captured verbatim in the stated-as slot. The model is told never to compute a date itself.

→

Closing Date is

not printed as a calendar date

The engine anchors on the agreement date from CA-01-03, 2016-05-04, and records that as an assumption.

→

Resolved

2021-05-04

Five years after the anchor. Confidence 1.00, with the chain, the anchor field and the assumption stored beside the value.

The same field on the other four documents

Doc 04 Two dates from one definition

2028-01-17 for the Amendment No. 7 commitments and 2026-12-17 for the non-extended ones, read from the attached agreement. A third instance, the swing loans, is kept as wording.

Doc 06 No defined term at all

The plan’s first step found nothing, so the fallback read Section 1.7, Maturity of Loans, and 2023-11-26 came from there with an exact citation.

Docs 01 and 02 The easy case

Printed in the definition: 2013-09-09 and 2027-02-01. Extracted, cited and checked against the closing date.

Deep dive: 45 commitment rows that add up

CA-02-05 on document 05. Every row is cited to its own line of the schedule, and the totals are checked against the facility size.

Schedule 1.01(a), as extracted: one record per lender and facility

Lender	Initial US	Initial Canadian	Initial European	Total Revolving
Citibank, N.A.	$28,700,000	$2,000,000	$9,300,000	$40,000,000
ING Capital LLC	$26,250,000	$1,750,000	$7,000,000	$35,000,000
… 8 more lenders				
Sum of the lender rows	$150,000,000	$10,000,000	$40,000,000	$200,000,000
CA-03-04 facility size, difference 0				**$200,000,000 ✓**

45

rows extracted, all green

10

lenders across 4 commitment columns

How the check works. The rows for each facility are summed in code and compared with the commitment the engine extracted separately in CA-03-04. The Total column matches to the dollar.

Two passages, one value

CA-03-12 across five agreements: the borrowing minimum and the notice period sit in different sentences, and each is cited.

Passage A · Section 1.01, character 242,545

“Minimum Borrowing Amount” means $250,000.

Passage B · Section 2.08, character 397,441

“not later than (i) in the case of each Borrowing of a Term SOFR Loan, 12:00 p.m. (Local Time) at least three (3) Business Days’ prior to the date of such Borrowing”

One record · document 04, Term SOFR Loan

rate_type Term SOFR Loan

minimum_amount 250,000 USD

increment_amount none stated

notice_business_days 3

The value quotes passage B; the part minimum_amount carries passage A as its own citation, 155,000 characters away. Confidence 1.00.

The same stitch on all five

Doc	Minimum	Notice	Amount · notice
01	$5,000,000	3 days	§2.02 · §2.02
02	$3,000,000	3 days	§2.02 · §2.02
04	$250,000	3 days	§1.01 · §2.08
05	$1,000,000	3 days	§2.02 · §2.03
06	$100,000	3 days	§1.5 · §1.6

The floating-rate loan on each document. Base-rate loans, and the Canadian-dollar loans on document 05, have their own records built the same way: 13 records, every part cited, all at 1.00.

The field is an object

Minimum, increment, currency, rate type and notice days in one value. The dictionary defines the shape; the model fills it from the located passages.

Each part quotes its sentence

The value quotes the passage it is mostly about. Every other part comes back as a verbatim passage aligned to the text; a missing one costs confidence.

Why it matters

A minimum without its notice period is half an answer. Stitching them is what an analyst does across two pages; here it is done once and checked.

Covenant ratios, read as ratios

CA-08-08 across five agreements: what each ratio is made of, which way it is tested, and the level it must meet.

Doc	Covenant	Direction	Required level	Numerator / denominator
01	Consolidated Leverage Ratio	Maximum	1.50 to 1.0	Funded Indebtedness less Subordinated / Consolidated EBITDA
01	Consolidated Fixed Charge Coverage	Minimum	1.35 then 1.50 to 1.0	Consolidated EBITDA / Fixed Charges
02	Consolidated Total Net Leverage	defined, not tested	none	Funded Indebtedness / Consolidated EBITDA
02	Consolidated First Lien Net Leverage	defined, not tested	none	First Lien Debt / Consolidated EBITDA
02	Fixed Charge Coverage Ratio	defined, not tested	none	Consolidated EBITDA / Fixed Charges
04	Fixed Charge Coverage Ratio	Minimum	1.00 to 1.00, springing	Consolidated EBITDA / Fixed Charges
05	Fixed Charge Coverage Ratio	Minimum	1.00:1.00 when triggered	Adjusted EBITDA / Fixed Charges
06	Total Indebtedness to Asset Value	Maximum	0.60 to 1.00	Total Indebtedness / Total Asset Value
06	Secured Indebtedness to Asset Value	Maximum	0.40 to 1.00	Secured Indebtedness / Total Asset Value
06	Adjusted EBITDA to Fixed Charges	Minimum	1.50 to 1.0	Adjusted EBITDA / Fixed Charges
06	Secured Recourse to Asset Value	Maximum	0.05 to 1.0	Secured Recourse Debt / Total Asset Value
06	Maintenance of Net Worth	Minimum	an amount, not a ratio	Tangible Net Worth

Two places, one record. The numerator and denominator come from the definitions article; the direction and the level come from the financial covenant section, each with its own citation. **Absence is stated.** Document 02 defines three ratios for its baskets and tests none, so the level is empty rather than invented.

Inside a composite filing

Four documents in one file, segmented on character offsets.

Wrapper	Cover	Contents	Attached agreement: articles, schedules, exhibits
`0 to 28,627`	`3,025`	`31,652 on`	`Article I begins at 42,440 and runs to the end`

character 0 — end of file

Segment before searching

Wrapper, cover and contents are marked off before any field is looked for.

Read only the agreement

Article I onward is the operative text — a cover date cannot pass as the execution date.

Keep the offsets

Every quote still cites its place in the original file, so a reader can find it.

Document 04 is the hardest shape in the set — and it runs end to end.

After the reply: what happens to every record

Six deterministic steps per record, each writing its result on the record. No value is changed by any of them; they only judge it.

1 Build
`build_record`
One record per field and instance; object parts parsed from quoted passages.

2 Align
`align_quote`
The quote is found in the text: exact, then fuzzy at 0.92, which costs 0.10. No match scores 0.

3 Conform
`conform`
Type and format held to the dictionary. A date in words is resolved, chain and assumption recorded.

4 Validate
`108 rules`
The field's own rule: pass, fail, or not applicable with a reason. A fail costs 0.40.

5 Cross-check
`69 checks`
The value against the partner field the dictionary names. A fail costs 0.20.

6 Verify & route
`verify · route`
A doubtful record gets a second opinion. Then the score, then routing by disposition.

Two records from document 05 through the six steps

Step	CA-02-05, commitment schedule, row "Total · Initial US"	CA-03-09, maturity date, Initial Revolving Loans
Build	Table row: Total, Initial US Commitments, $150,000,000	Value null; stated_as "five years after the Closing Date"
Align	Exact, Schedule 1.01(a), one schedule line	Exact, Section 1.01, the definition
Conform	Table with the four columns the format names: conformant	Resolved to 2021-05-04; the assumption recorded
Validate	Pass: the 10 lender rows sum to $150,000,000	Not applicable: no printed closing date to test against
Cross-check	Pass: equals the CA-03-04 commitment	Pass: CA-06-01 is a single balloon at the Maturity Date
Verify, score, route	No second opinion. Score 1.00. Auto-accept eligible	No second opinion. Score 1.00. Routed: rests on an assumption

Where the POC stands

Built and working

- ✓ **Trace** — all nine stages per field, with the sections read
- ✓ **Cost accounting** — per stage, per field, per document
- ✓ **Review sheets** — one card per field, sign off or send back
- ✓ **Annotation tool** — records beside the text they came from
- ✓ **Human in the loop** — red and amber routed to a reviewer
- ✓ **Confidence score** — starts at 1.00; only the engine's own failed checks lower it
- ✓ **Validation** — each field against its own rules
- ✓ **Cross-checks** — each field against the fields around it
- ✓ **Prompt and schema versions** — stamped on every record

Not in scope yet

Golden dataset

No measured accuracy figure — confidence here is reasoned, not counted.

Entity master

Nothing reconciled against an external system of record.

Amendment-only filings

Composite and restated filings only, so far.

What a document costs

A full run, all 138 fields, one document.

Today, on Opus 5	$8 – $14
With the tiered model plan	$6 – $7.50
Tiered, through the Batch API	$3 – $3.75

$1.5 - $2

Target Spend post optimization

Against analyst hours per document, the arithmetic is not close.

Scaling up: accuracy, cost and time

What production at 1,000 agreements a month needs, and the order to build it in.

ACCURACY

A golden set, then learning

about 30 agreements, about 4,000 fields labelled in the tool

- **1. Errors located.** Accuracy per field and per agreement type replaces today's consistency score, so effort goes to the weak fields.
- **2. Green made trustworthy.** Measured correctness sets the bands: if green is 99% right it skips review, and reviewers take the ambers and reds.
- **3. Fixes sent to the right stage.** Reason codes point to locate, prompt, keying, definitions, normaliser or the dictionary, found systematically and ranked by impact.
- **4. Corrections become examples.** Reviewed passages and their answers are shown for the same field when wording is unusual; no retraining.
- **5. Gains compound safely.** Every change, a prompt, a validator or a model, is scored on all labelled fields before it ships.
- **6. Checks tuned on evidence.** Validators and cross-checks are weighted by the real errors they catch, not by judgement.

COST

From $4.81 to about $1.50 a document

about $4,800 to $1,500 a month at 1,000

- **Batch API for anything not urgent.** Half price on every token: $4.81 becomes $2.40.
- **Right-size the model.** Verification, $1.31 of the $4.81, is a yes-or-no judgement: Haiku 4.5 costs a quarter as much. Simple extraction groups on Sonnet 5 at half price; Opus 5.5 kept for tables and stitched fields.
- **Jev by TypeSafe.** Typed verdicts with probabilities, suited to verification and routing, not extraction. The shadow harness is built; it needs a key, and pricing is unpublished.
- **Open-weight or local models.** No token fee, but GPUs and operations. They fit classification, table parsing and first-pass checks, and keep data in-house; at about $1,500 a month they pay only for data residency or far higher volume.
- **Send fewer tokens.** Tighter passages, table rows parsed in code, and caching (Opus 5.5 reads its cache at 5% of the input price).

TIME

From 11 minutes to about 3 a document

1,000 a month in under an hour a working day

- **Today every call waits for the last.** Extraction and verification calls a document run in sequence: 1,000 documents would take 183 hours of one worker.
- **Parallel calls within a document.** The call groups are independent: six at a time bring extraction to about 2 minutes, and eight verifications at a time take under a minute.
- **Several documents at once.** The batch page runs one at a time today; multiple workers clear 100’s of agreements
- **Batch API for volume.** Usually back within few hours, 24 hours at worst, at half price; real-time kept for urgent documents.

Order of work: the golden set first. It is the yardstick that lets every cheaper model and every faster run go to production without losing accuracy.

Dynamic effort: pay for thinking only where it helps

One call answers a group of fields, 138 fields in multiple calls, so effort is set per call group, and from evidence.

What the three levels mean

Low	a quick look: fewest tokens, fastest, cheapest; for clear-cut questions
Medium	the balance, and Opus 5.5's default; what extraction uses today
High	a careful look: checks itself and weighs alternatives; slower, costs more

The thinking is billed as output: document 05's CA-02 cost $0.65 at medium, and about $0.80 to $1.00 at high (estimated).

Why the dictionary's ratings are not enough

63	61	14
fields rated High	fields rated Medium	fields rated Low

A call runs at the level of its hardest fieldd. Setting effort from the ratings would put almost every call on high: more cost, no saving.

1 Start at medium

Every call group at medium, as today: the proven baseline and the one every estimate is built on.

2 Escalate when a check fails

A group whose answer fails a validator, a cross-check or a completeness check, or returns fewer rows than the document shows, is re-run at high. One or two groups a document, not 36; document 02's missed Holdings link is the case. Needs no golden set: the recommended first step.

3 Learn the levels from the golden set

Per call group, measure accuracy at low, medium and high and keep the cheapest level that meets the target: identification probably low, schedules and covenant ratios perhaps high. The same table can name the model, so effort and model tiering are one setting.

What the engine needs: a settings table per call group; the escalation rule in the extraction step; the level recorded on every call and record, so cost and time can be measured per level; and an “auto” option on the batch page.

Trust, not just output

A number without a place to point at in the document is a guess. Every value the pipeline produces arrives with its evidence attached.

How the confidence score is calculated

A stated policy, not a measured probability.

CONDITION ON THE RECORD	EFFECT	WHY
No verified citation	**score = 0.00**	unusable, whatever else
...unless a second opinion found one	**score = 0.60**	usable, not clean
Otherwise	**start at 1.00**	
Quote aligned only by fuzzy match	− 0.10	text differs slightly
Validation rule failed	− 0.40	contradicts its rule
Cross-check failed	− 0.20	partner field disagrees
Second opinion contradicts	− 0.50	the passage says no
Second opinion: not found	− 0.20	could not confirm
Extraction needed a schema retry	− 0.05	first answer malformed

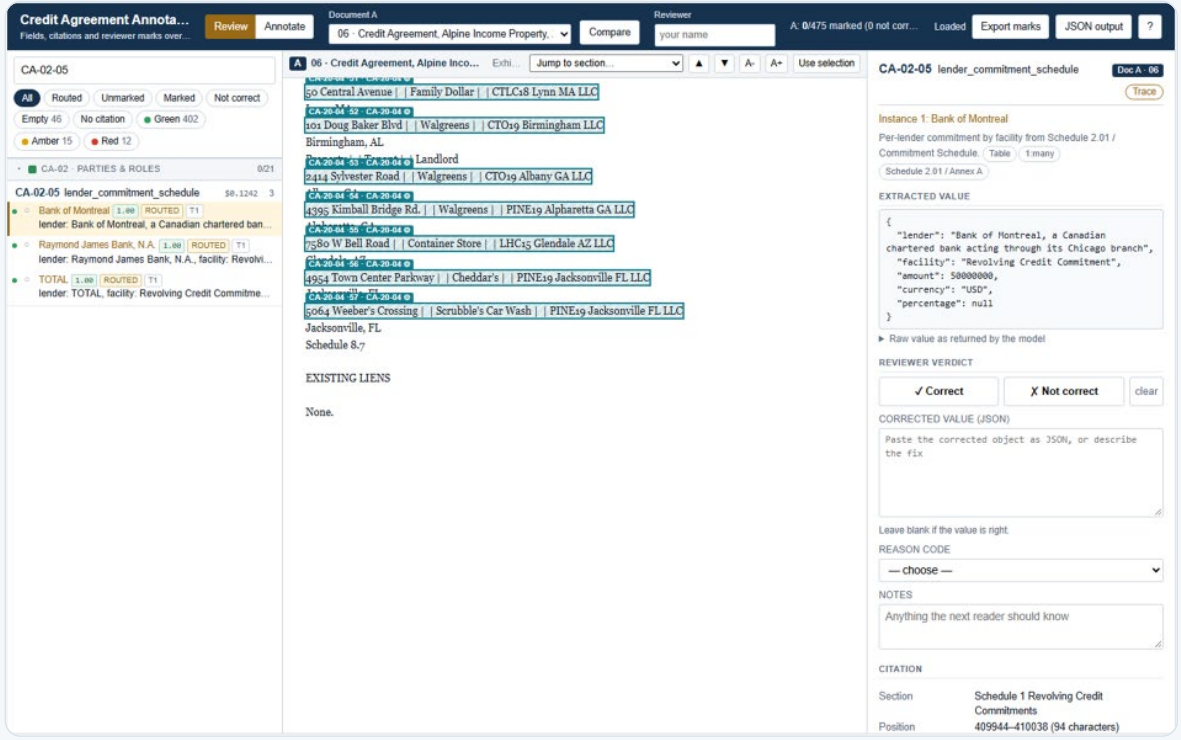
Applied in this order; the result is clamped to 0 and 1.

- **Green** 0.95 and above
  Cited, every check passed. Auto-accept eligible.
- **Amber** 0.60 to 0.94
  One thing to look at: a fuzzy quote or a failed check.
- **Red** below 0.60
  No citation, a contradiction, or two failures.
- **Empty** no value
  Not stated in the filing; the reason is recorded.

Uncalibrated by design. No weight is fitted to an observed error rate, because there is no labelled set yet. A high score means nothing the engine could check came back wrong — not that the value is correct.

Review without opening the filing

Every record beside the passage it came from, with the reviewer's verdict captured in place.



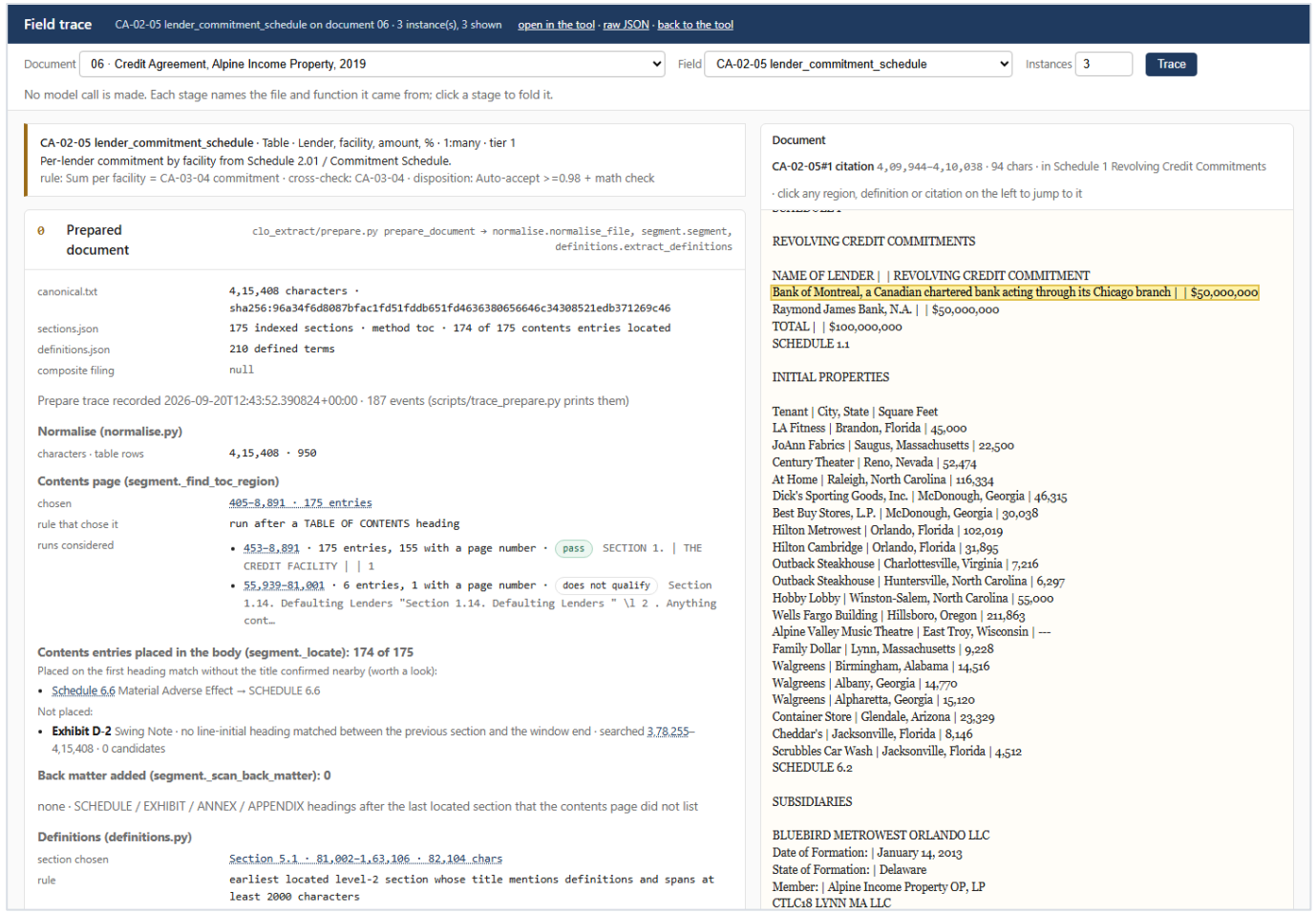
Document 06, lender commitment schedule — record list, cited passage, review panel.

- **Filter by band.** Green, amber, red and empty counts sit on the record list; one click shows the reds.
- **Click a record, see the page.** The document pane scrolls to the highlighted citation.
- **Mark it.** Correct, not correct, a corrected value and a reason code, saved per reviewer.
- **Compare two documents** side by side, field for field.
- **Trace it.** Every record links through to the stage-by-stage trace page.

This is the human in the loop on the status slide, as an actual screen rather than a claim.

Transparency: one field, walked through every stage

The trace page shows how a value was produced, with the document beside it and each decision clickable

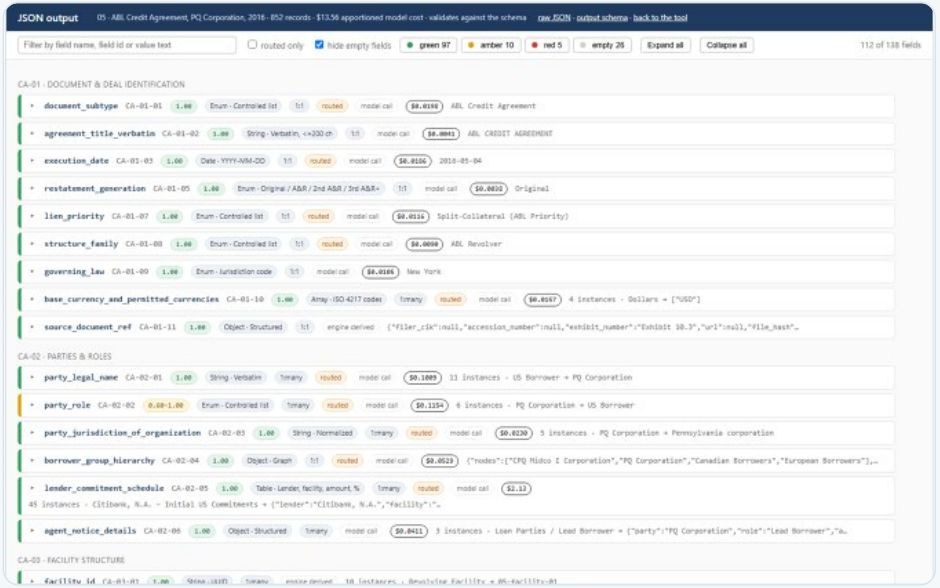


Trace of CA-02-05 on document 06, with Schedule 1 highlighted in the document panel

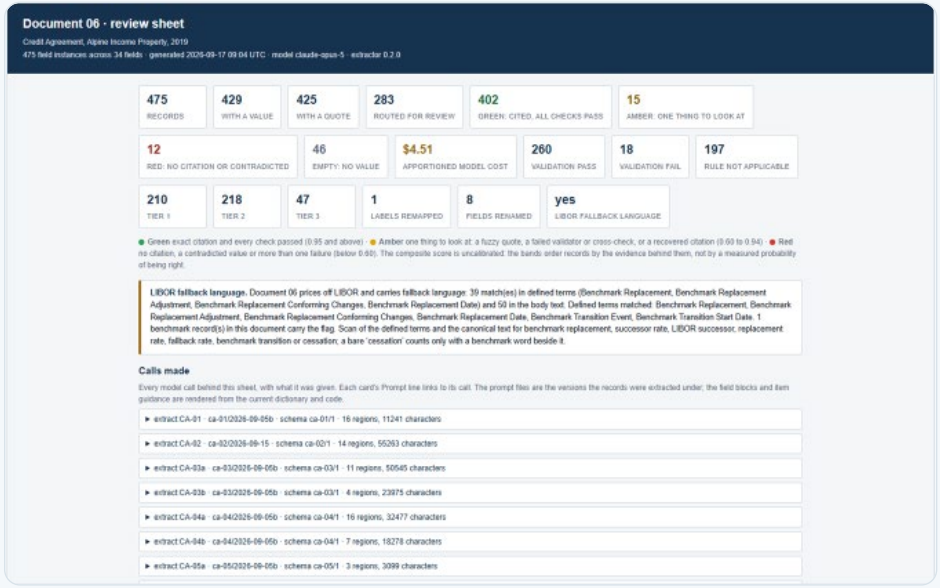
Nine stages, each naming its file and function

0. Prepared document, with every decision the prepare stage made
1. Locate: the plan, the passages found, and which fields asked for them
2. Prompt as assembled
3. The call: tokens, cost, call id
4. Records built, quote in context
5. Conformance: dates resolved, formats checked
6. Validator and cross-check, re-run live
7. Confidence and routing
8. Cost apportioned to the field

The same records, three ways



JSON output — every field typed and validated against a published schema.



Review sheet — one page per document: counts, cost, every record in context.

For systems

A JSON Schema keyed by field name; every export validates against it.

For reviewers

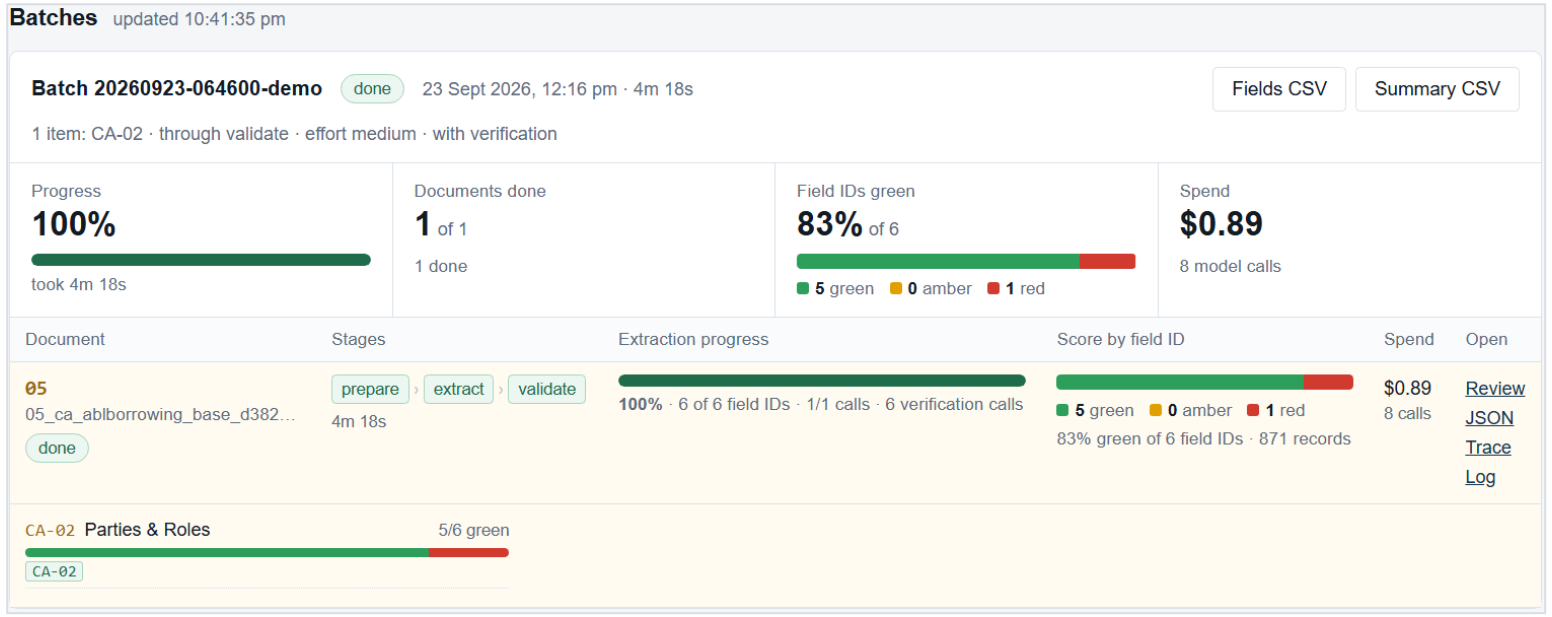
Review sheets from a file share; corrections feed the golden set.

For management

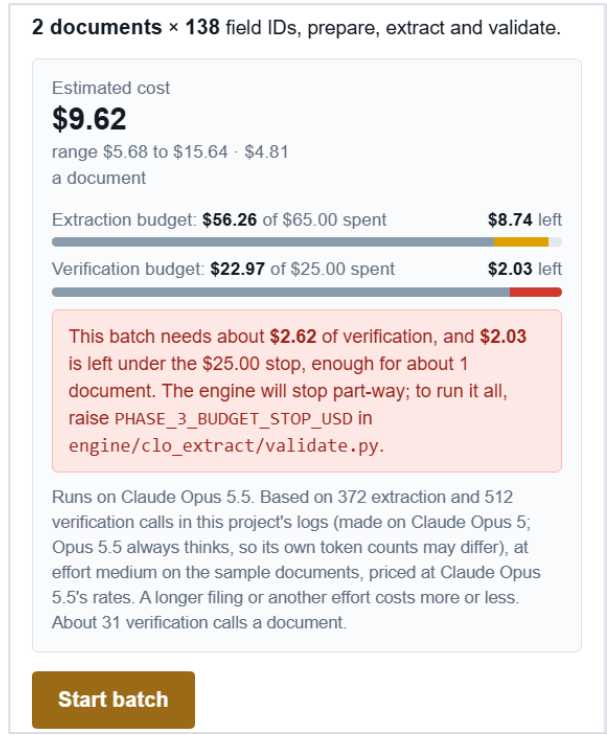
Spend per document and cost per field — and no accuracy figure yet.

Batch extraction: upload, price, watch, score

A page in the tool that runs one or many filings through the engine, prices the batch before it starts, and scores every field when it ends.



A finished batch: document 05, item CA-02, 5 of 6 fields green in 4 minutes for $0.89; the row opens to each item



Before two documents start: cost and budget left

Upload or re-run

Drop one or many filings, HTML, PDF or text, or pick documents already on disk. Choose the items, the effort, verification, and whether to run through prepare, extract or validate.

Priced before it starts

Cost with a range, from 884 logged calls re-priced at Opus 5.5 rates, and the headroom under the engine's two budget stops. A paid run asks to be confirmed; with no API key it does not start.

Watched while it runs

Each document's stages, progress as the share of its field IDs whose call has returned, the time left at the pace so far, and spend. Cancel stops the running stage.

Scored when it ends

Green, amber, red and missing by field and by item; links into review, JSON, trace and the log; a per-field and a per-batch CSV. The state is kept on disk, so a restart loses nothing.

Time per document	Typical	Range
Validate, the full run	11 min	9 to 16 min
Extract only	9 min	8.5 to 11 min
Prepare only	1 sec	

What comes next

01

External cross-checks

Filing metadata, the certificates, and an entity master to reconcile against

02

Amendment-only filings

The ones that change an agreement without restating it

03

The tiered model plan

Cheaper models on the stages that do not need judgment

04

A golden set

So accuracy can be measured rather than reasoned about

The fourth is the one that turns a demonstration into a result.

Q&A

Appendix

Opus 5 or Opus 5.5? One item, three runs

CA-02 Parties & Roles on document 02: Opus 5 on 5 September, then Opus 5 and Opus 5.5 today on the same prompts.

	Opus 5, 5 Sep	Opus 5, today	Opus 5.5, today
Records: green / amber	20: 13 / 5	21: 19 / 0	21: 19 / 0
Quotes that match the text	13 of 18	19 of 19	19 of 19
Verification calls needed	5	0	0
Party roles found	9	10, one a group	11
Morgan Stanley as arranger and bookrunner	missed	missed	found
Company hierarchy links	2	3, one a group	1
Jurisdictions	3	3	2
Legal names	5	5, same	5, same
Extraction call	$0.23	$0.25	$0.22
Output tokens	1,601	2,079	2,858

Same document, same item; the 5 September run used that day's prompts, the two runs today the current ones. Lender commitments and agent notice details were empty in all three.

The engine, not the model

On today's engine Opus 5 also quotes every part exactly: the five ambers and five verification calls of 5 September are gone for both models. That gain is three weeks of prompt and locator work.

Better on Opus 5.5

All eleven roles, including Morgan Stanley as a joint lead arranger and bookrunner, which Opus 5 missed twice. No groups entered as parties. The extraction call 14% cheaper, despite 37% more output tokens from thinking.

Worse on Opus 5.5

Dropped the Holdings link, DiscoverOrg Midco as parent of the Borrower, which Opus 5 kept twice, and the jurisdiction of Zebra Acquisition Corporation. Both records still scored green: no check tests completeness.

Verdict: Opus 5.5 is slightly better on the field that matters most, slightly worse on the hierarchy, and cheaper. One item on one document is too small a sample to decide the switch; the golden set is the test. Neither miss lowered the score, so completeness checks, starting with the Holdings link, would be the first to add.

What every value carries

Five things, attached to each of the 138 fields.

01

A verbatim quote, aligned to the character

02

A validator result against the field's own rules

03

A cross-check against the fields around it

04

A second opinion, wherever anything disagreed

05

A confidence band, so a reviewer knows where to look

Nothing is asserted without somewhere in the document to point at.

Fields that need the same text share a call

Item CA-03, one structured call: nine regions, 65,352 characters, six fields — and each field is still charged only for the regions it asked for.

REGION	CHARS	ASKED BY
Recitals (preamble block)	1,393	1 field
Definition: Aggregate Revolving Commitments	234	2 fields
Definitions: Revolving Commitment, Loan, Note	690	2 fields
Definitions: Swing Line — five terms	663	2 fields
Definition: Total Revolving Outstandings	141	2 fields
Sections 2.01–2.04 Commitments, Borrowings, LCs, Swing Line	45,306	5 fields
Sections 2.06–2.07 Termination, Repayment	2,719	3 fields
Section 4.02 Obligations Unconditional	3,886	1 field
Article V · Sections 5.01–5.02 Conditions Precedent	10,320	1 field

9 → 1

regions gathered into a single structured call for the item

45,306

characters in the biggest region — five fields wanted it, it went once

3.9×

less text than sending each field its own regions

Prompts are assembled

System prompt

One versioned file for every call: nine rules in order. Extract only what the text supports; no quote, no value.

Document context, cached

The passages locate chose, verbatim with section ids and offsets. Cached: a retry rereads at a tenth of the price.

Instructions, from the dictionary

Every field rendered from its dictionary row: type, cardinality, definition, format, allowed values, location.

Schema, as a constraint

Sent as the structured-output format, not as prose. Never read as advice; enforced as the answer is generated.

The other three calls

1. **Classify**

   One call per document over its opening text and first section titles — an amendment is an amendment however much it attaches.

2. **Recall**

   Only when a count taken from the document disagrees with the first pass; asks for the missing instances only.

3. **Verify**

   One call per doubtful record: “is this value supported by this text?” Four verdicts, parsed from the first line.

Every prompt file carries its date in its name, and every record is stamped with the prompt and schema versions it came from.

The cross-reference graph

Built once per document, without a model: every Section, Schedule, Exhibit, Article and Annex pointer, resolved against the section index.

One definition, two pointers — document 01

Section 1.01, definition — “Swing Line Loan Notice” means a notice … **Section 2.04(b)** … substantially in the form of **Exhibit B**

Section 2.04(b) — Swing Line Loans
`characters 151,449–161,199`

Exhibit B — Form of the notice
`characters 440,136–441,690`

How the graph is used

Definitions

A defined term is read together with the terms it relies on, one level down.

One-hop follower

Behind a flag, and off by default: pointers inside a field's passages are followed and their targets read too.

Trace page

Every field shows its pointers, where they resolve, and which the follower would read.

How the schema is enforced

One schema, generated from the dictionary, applied at four points. A value is never coerced into shape.

1

Generated from the dictionary

One typed model per field group. Closed enums, real calendar dates, no extra keys.

2

Enforced while decoding

The schema travels with every call, so the answer is held to shape as it is generated.

3

Checked again on arrival

Validated in code, catching what a grammar cannot: 30 February. One retry, then recorded as failed.

4

Validated before publication

Every export is checked against a published JSON Schema before it is written.

Three things around the schema

Wording is kept, not lost

“Five years after the Closing Date” sits beside the typed value; resolved to 2029-09-03, chain recorded.

A retried call is marked

Its records carry the flag and loses confidence — visible, and slightly less trusted.

The schema is versioned

Each record names the version it was extracted under

Deep dive: borrowing minimums and notice periods

CA-03-12. One object per rate type, across two currencies, with every part cited to the sentence that states it.

Document 05, seven instances from Section 2.02

Rate type	Currency	Minimum	Increment	Notice
LIBO Rate Revolving Loans	USD	$1,000,000	$100,000	
ABR Revolving Loans	USD	$100,000		
CDOR Revolving Loans	CAD	CAD 500,000	CAD 100,000	
Canadian Prime Rate Revolving Loans	CAD	CAD 100,000		
LIBO or CDOR, in Dollars				3 business days
LIBO or CDOR, other currencies				4 business days
ABR or Canadian Prime Rate				same day

`CITATION · document 01`

Section 2.02, characters 118063–118234. Engine, first pass, aligned exact.

> “Each Borrowing of … Eurodollar Rate Loans shall be in a principal amount of $5,000,000 or a whole multiple of $1,000,000 in excess thereof.”

`notice_business_days · 117470–117820`

> “Each such notice must be received by the Administrative Agent … three Business Days prior to the requested date of any Borrowing.”

Why this field is hard, and what changed

The parts come from different sentences. A minimum is stated in one place and the notice period in another. Document 01’s three-day notice for Eurodollar loans had a value but no citation of its own.

Now every part carries its passage. Object and table fields return a quote per part, each aligned to the text. A part whose quote cannot be found lowers the score by 0.05 and is listed on the record.

Every item, end to end



01 Deal identification · 11

02 Parties and roles · 6

03 Facility structure · 12

04 Pricing and interest · 10

05 Fees · 6

06 Amortization · 4

07 Prepayments · 7

08 Financial covenants · 9

09 Negative covenants · 9

10 Affirmative covenants · 7

11 Borrowing base (ABL) · 10

12 Collateral · 6

13 Intercreditor · 5

14 Conditions precedent · 3

15 Representations · 4

16 Events of default · 6

17 Key defined terms · 8

18 Assignments, voting · 7

19 Agent provisions · 4

20 Exhibits, schedules · 4

Nothing hides behind the interface

Four ways to look at the same run.

Annotation tool

Every record with its band chip, beside the passage it came from

JSON viewer

The raw output, field by field, exactly as it sits on disk

Review sheets

One card per field for a human to sign off or send back

Trace page

One field walked through all nine stages with the document open beside it — down to which sections the model actually read, and what each of them cost.

How the passages for a field are found

A fallback step fires only when the steps before it found nothing.

1 The plan runs in order

128 of the 138 fields carry a plan: an ordered list of steps — cover page, preamble, a defined term and its dependencies, matching sections, an article.

2 Every step adds passages

A step never replaces what an earlier one found; a field located by a term and a section reads both. Each passage is stamped with its step.

3 A fallback fires on empty

21 plans end in a fallback. It is skipped when anything was found, and runs only when no term, section or article matched.

4 Merge and cap

Passages closer than 400 characters merge. None exceeds 120,000 characters and no item's passages exceed 200,000 in total.

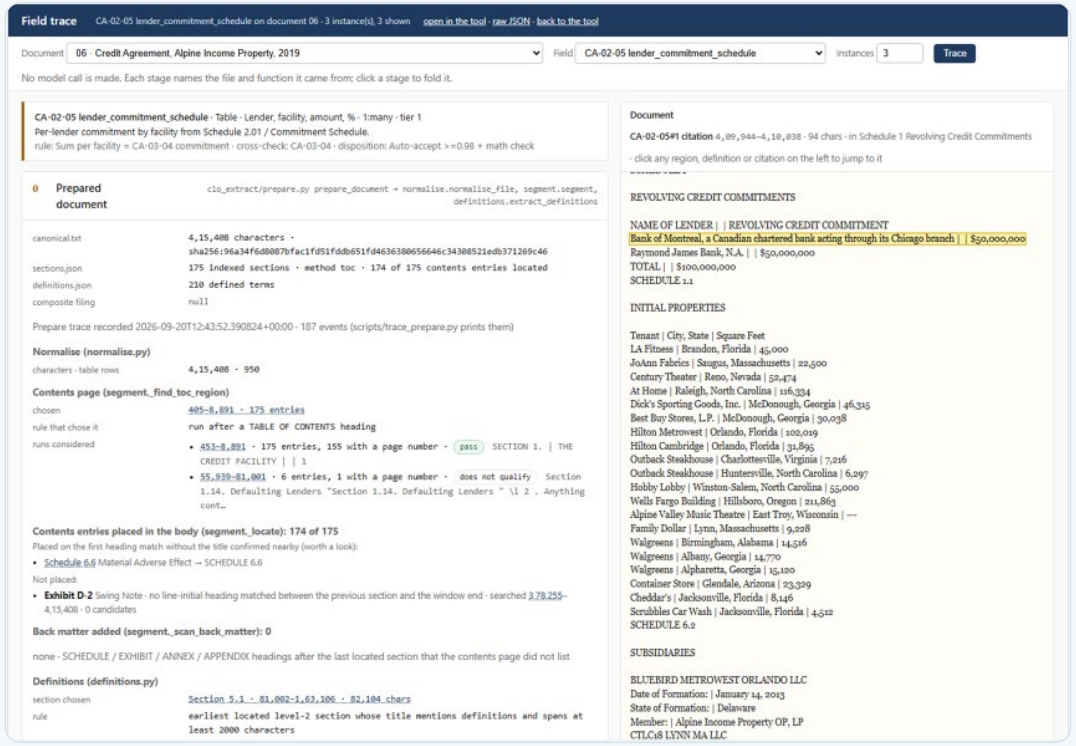
The three fallback kinds

Step	Fields	What it reads when the plan found nothing
`fallback_sections`	10	the sections whose titles match a pattern for the field
`fallback_sections_wide`	10	the same, but up to 24 sections and the whole item budget
`fallback_article`	1	the whole article whose title matches, heading to the next

No model decides where to look. A model-driven locator exists in the code and stays unused. A field added to the dictionary without a plan is located from its own Typical location cell by a small grammar.

One field, walked through every stage

How a value was produced, with the document beside it and every decision clickable.



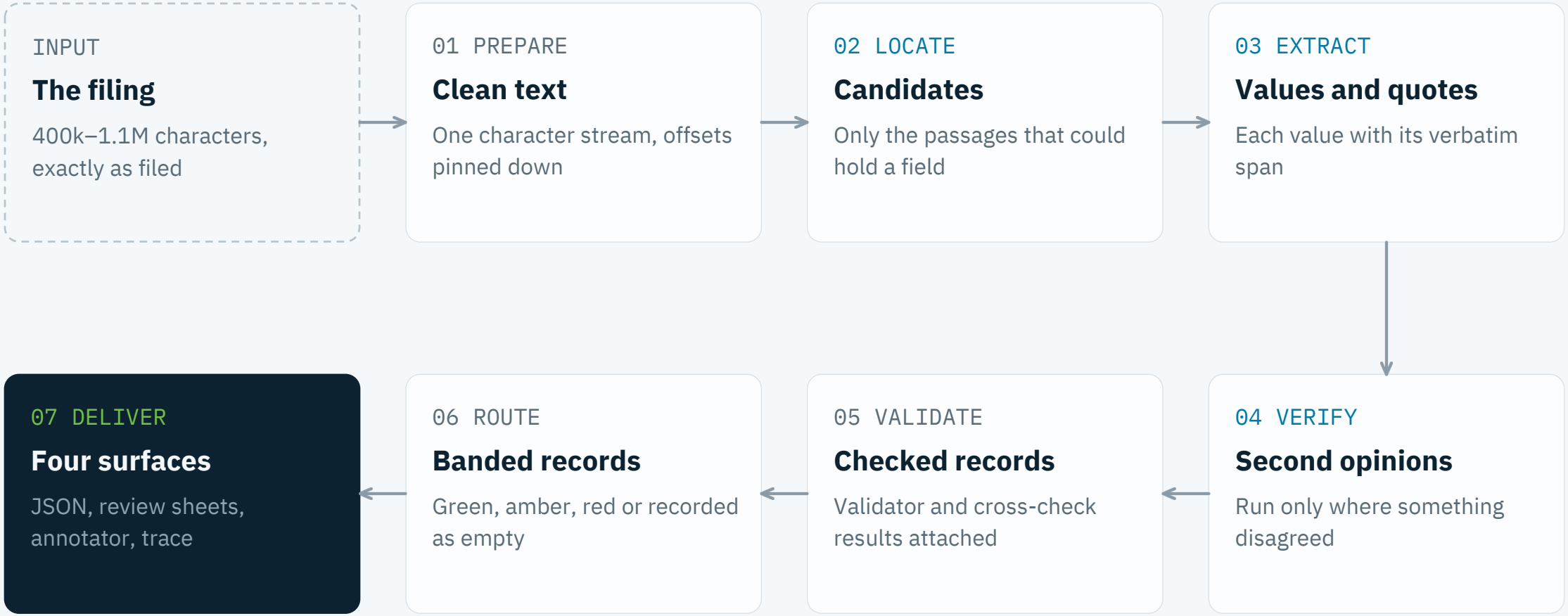
CA-02-05 on document 06, with Schedule 1 highlighted in the document panel.

Nine stages, each naming its file and function

0. Prepared document, with every decision the prepare stage made
1. Locate: the plan, the passages found, and which fields asked for them
2. Prompt as assembled
3. The call: tokens, cost, call id
4. Records built, quote in context
5. Conformance: dates resolved, formats checked
6. Validator and cross-check, re-run live
7. Confidence and routing
8. Cost apportioned to the field

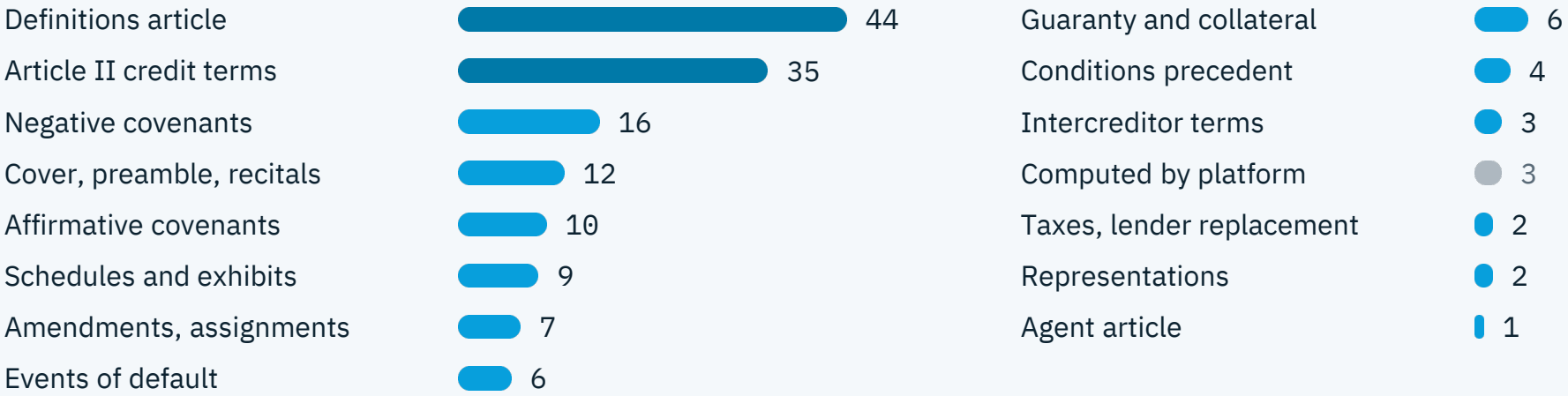
What a filing turns into

model call · deterministic



Which sections the fields come from

Across all 20 items. One field often draws on more than one section.



160

section reads across the 138 fields — many fields need more than one

49%

come from two sections alone: definitions and Article II

15

sections in all — ten of them supply fewer than ten reads each

Priority and tier by item

Fields per item under each grading.

ITEM	P0	P1	P2	T1	T2	T3
CA-01 Identification	7	4	0	5	5	1
CA-02 Parties	2	3	1	1	4	1
CA-03 Facilities	5	6	1	11	0	1
CA-04 Pricing	4	5	1	8	2	0
CA-05 Fees	2	2	2	3	2	1
CA-06 Amortisation	2	1	1	3	1	0
CA-07 Prepayments	3	3	1	3	3	1
CA-08 Financial covenants	8	1	0	5	4	0
CA-09 Baskets	6	3	0	6	3	0
CA-10 Reporting	3	4	0	3	4	0
CA-11 Borrowing base	8	2	0	7	3	0
CA-12 Collateral	2	4	0	2	4	0
CA-13 Intercreditor	2	3	0	2	3	0
CA-14 Conditions	0	2	1	0	2	1
CA-15 Representations	0	2	2	0	2	2
CA-16 Defaults	5	1	0	5	1	0
CA-17 Definitions	7	1	0	4	4	0
CA-18 Voting, assignment	2	3	2	2	3	2
CA-19 Payments and tax	0	1	3	1	0	3
CA-20 Exhibits, schedules	3	1	0	3	1	0
Total 138 fields	**71**	**52**	**15**	**74**	**51**	**13**

123 of the 138 fields are P0 or P1, and 125 are T1 or T2.

What it covers today

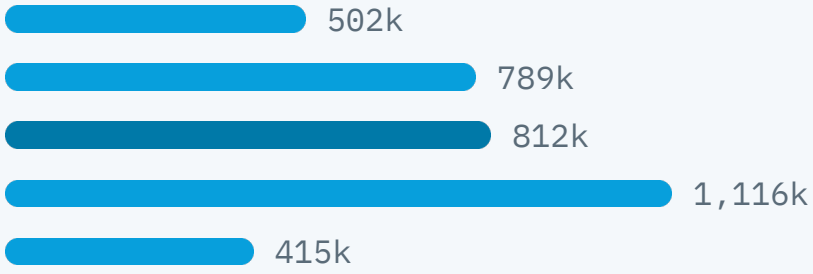
20
extraction items, all built

138
fields per document

5
documents in the counted set

690
field results, every one inspectable

DOC	FILING	CHARACTERS
01	Credit Agreement, 2010, LIBOR era	502k
02	Second Lien Credit Agreement, 2020	789k
04	Amendment No. 7, restated agreement attached	812k
05	ABL Credit Agreement, PQ Corporation, 2016	1,116k
06	Credit Agreement, Alpine Income Property, 2019	415k



3.63 million characters in all, from 415k to 1.1M — and 04 is the composite amendment filing.

Five ways a value is obtained

Every record says which, and why. 2,686 records on documents 01, 02, 04, 05 and 06.

Model

2,155 records · 118 fields

The code finds the passages; the model returns typed values with a quote for every part; the code places each quote in the text, enforces the schema and normalises.

For example: Lender commitments, margin grid, covenant ratios

Model, second pass

28 records · 5 fields

When a list looks shorter than the document, counted in code (facility-like terms, schedule rows), one more call asks only for the missing entries.

For example: Facility names, covenant categories

Model + engine

6 records · 2 fields

The model returns the wording and the code resolves it: “five years after the Closing Date” becomes 2021-05-04, with the chain and any assumption on the record.

For example: Closing date, maturity date

Engine only

300 records · 10 fields

No model call. Read from the prepared filing or computed from other fields, with exact citations where there is a passage.

For example: Defined terms, schedule inventory, balloon

Absent

197 records

The engine looked and found nothing: the model reported the field not stated, or no passage was found. Kept with the reason, never left blank.

For example: Credit spread adjustment, PIK

The ten engine-only fields and their sources

Source	Fields
Defined-term dictionary	Term names, definitions, MAE definition, dependency graph
Contents page and section index	Exhibit and schedule inventories
Other fields' values	Facility id; amortisation rate and balloon
The file itself	Source document reference: hash, size, filing

Code around every model value

- **Before:** the code chooses the passages; a model search over section titles exists as a last resort and has never been needed.
- **After:** schema check with one retry, quote placement, format and date conformance, client vocabulary, validators and cross-checks.
- **Second opinion:** a low-effort model call on records a check disagrees with; it recovered 21 citations.

Appendix: the code behind each stage

50 Python files, 27,444 lines, 643 test functions. Every file belongs to one stage; the next eight slides take each stage apart.

0 Prepare	1 Locate	2 Extract	3 Align, conform	4 Validate, verify	5 Score, route	6 Outputs
7 files · 2,726 lines · 60 tests	2 files · 1,797 lines · 33 tests	5 files · 2,676 lines · 54 tests	7 files · 2,532 lines · 90 tests	3 files · 6,684 lines · 111 tests	5 files · 1,300 lines · 58 tests	8 files · 3,289 lines · 29 tests

`normalise.py`
`segment.py`
`definitions.py`
`references.py`
`composite.py`
`prepared.py`
`prepare.py`
	`locate.py`
	`location_grammar.py`
		`classify.py`
		`extract.py`
		`key_terms.py`
		`amortisation.py`
		`inventory.py`
			`align.py`
			`conformance.py`
			`rekey.py`
			`vocab.py`
			`vocab_remap.py`
			`field_rename.py`
			`libor_fallback.py`
				`validate.py`
				`validators.py`
				`verify.py`
					`confidence.py`
					`tiers.py`
					`route.py`
					`provenance.py`
					`costing.py`
						`common.py`
						`template.py`
						`review.py`
						`corrections.py`
						`json_output.py`
						`metrics.py`
						`vocabularies.py`
						`__main__.py`

Foundation, used by every stage: `contract.py` the record every stage reads and writes; `spec.py` the client's 138-field dictionary as typed schemas; `llm.py` the only code that talks to the model; `trace.py` a timeline of every run. 4 files, 2,723 lines: spec and llm on the Extract slide, contract and trace on the last.

Running it and seeing inside it: the annotation tool and batch page, the batch runner, the cost and time estimates, the field trace, and the golden-set helpers. 7 files, 3,691 lines, 41 tests. A further 167 tests check each item end to end: its schema, its derived fields and its checks.

Stage 0, Prepare: the filing becomes addressable text

Deterministic and free: the filing becomes text with stable offsets, a section index, the defined terms and the cross-references.

1 normalise.py — 671 lines

Filing to canonical text

- **normalise_file()** HTML or PDF, chosen by the file type
- **normalise_html()** renders the filing to text with a map back to the source
- **normalise_pdf()** reads a PDF page by page, dropping running headers and footers
- **hash_text()** the document's fingerprint

2 segment.py — 694 lines

The section index

- **segment()** finds the contents page and places every entry in the body
- **parse_toc_line()** reads one line of the contents page
- **find_body_start()** where the operative text begins

3 definitions.py — 473 lines

The defined terms

- **find_definitions_section()** picks the definitions article
- **parse_definitions()** every “Term” means … entry
- **build_dependencies()** which terms each definition uses
- **find_cycles()** definitions that refer to each other

4 references.py — 254 lines

The cross-reference graph

- **build_reference_graph()** resolves every “Section 2.04(b)”, schedule and exhibit to the text it names, and each section's defined terms

5 composite.py — 147 lines

Amendments with the agreement attached

- **find_attached_agreement()** where the attached agreement begins
- **probe_attached_agreement()** the cues it matched, for the trace

6 prepared.py — 258 lines

Reading it back

- **PreparedDoc** the prepared files, loaded for every later stage
- **pseudo_sections()** names the cover, preamble, recitals and signature pages
- **signature_span()** the agreement's own signature pages

7 prepare.py — 229 lines

The command

- **prepare_document()** runs 1 to 5 on one filing and writes its files and trace
- **main()** python -m clo_extract.prepare

Tests: test_normalise, test_normalise_running_lines, test_segment, test_definitions, test_references, test_composite, test_prepare, test_prepare_trace, test_prepared_sections, test_signature_region: 60

Stage 1, Locate: code chooses what the model reads

For every field, the passages it lives in, from the section index, the definitions and the dictionary's “typical location”. No model call.

1 locate.py — 1,252 lines

A plan per field

- **locate_field()** the regions for one field, from its plan
- **preamble_region(), signature_region(), credits_regions()** the standard places a field lives
- **term_regions()** a defined term and the terms it depends on
- **sections_matching()** sections whose title fits the field
- **merge_regions()** joins overlapping passages and caps their size

2 location_grammar.py — 545 lines

Plans from the dictionary

- **parse()** reads the dictionary's “typical location” cell into steps: anchor, term, article, schedule
- **resolve()** turns the steps into regions for fields without a hand-written plan
- **score()** measures how often the located passages hold the labelled answer

3 locate.py: follow_references() — optional

One hop along the references

- **follow_references()** adds the sections a located passage points to, within a budget per call, off by default

4 locate.py: llm_locate() — last resort

The model, only if the code finds nothing

- **llm_locate()** one small call over the section titles; it exists and has never been needed

Tests: test_locate, test_locate_follow, test_location_grammar: 33

Stage 2, Extract: where the model is used

One structured call per call group over the located passages, a second pass when a list looks short, and the fields that are computed in code.

1 classify.py — 164 lines

Is it a credit agreement?

- **classify()** one short, low-effort call on the opening pages
- **route()** extract it, or set it aside with the reason

2 spec.py — 1,807 lines

The dictionary as code

- **load_fields()** the client's 138-field workbook
- **build_item_model()** a typed schema the model must answer in
- **call_groups()** splits large items into several calls
- **render_field_instructions()** each field's definition and format, for the prompt

3 llm.py — 702 lines

The only door to the model

- **LLMClient.structured()** the call, checked against the schema, retried once
- **LLMClient.cited_text()** the verification call, with citations
- **CostLog** every call's tokens and dollars, and the budget stop

4 extract.py — 1,700 lines

Extraction

- **run_document()** classify, locate and extract every call group
- **extract_item()** one call over the located passages
- **compute_anchors(), recall_call()** counts in code, and a second pass when a list looks short
- **build_record()** a record, its quote placed in the text

5 key_terms.py — 272 lines

Defined terms, no model

- **select_key_terms()** the key terms of the document
- **dependency_graph()** what each term depends on

6 amortisation.py — 208 lines

Arithmetic, no model

- **annual_rate()** amortisation rate from the schedule rows
- **balloon()** principal less every scheduled instalment

7 inventory.py — 332 lines

Exhibits and schedules, no model

- **inventory()** from the contents page, the section index as fallback
- **populated()** whether a filed schedule carries data

Tests: test_classify, test_spec, test_llm, test_extract, test_key_quotes: 54; and 167 per-item tests, test_ca04b to test_ca18_ca20

Stage 3, Align and conform: every answer checked

Code checks what the model returned: quotes found in the text, dates resolved, formats enforced, names and labels in the client's terms.

1 align.py — 258 lines

Quote to text

- **align_quote()** finds the model's quote in the text: exact, then whitespace-normalised, then close
- **verify()** the quote must equal the text at its offsets

2 conformance.py — 786 lines

Formats and dates

- **apply_top_level()** resolves “five years after the Closing Date” to a date
- **check()** each value against the dictionary's data type and format
- **annotate()** both, for every record

3 rekey.py — 124 lines

The right instance keys

- **rekey_maturities()** a maturity keyed by its term moves to its facility

4 vocab.py — 516 lines

Controlled lists

- **the eight vocabularies** the values each list field may take
- **client_enumerations()** the client's own lists, read from the workbook

5 vocab_remap.py — 306 lines

The client's labels

- **apply_to_records()** relabels five list fields in the client's words, with the history kept

6 field_rename.py — 278 lines

The client's field names

- **apply_to_records()** the client's names for the CA-01 and CA-02 fields, with the history kept

7 libor_fallback.py — 264 lines

The LIBOR flag

- **scan()** fallback language in the definitions and the text
- **annotate_records()** flags the legacy LIBOR records

Tests: test_align, test_conformance, test_rekey, test_field_names, test_vocab_update: 90

Stage 4, Validate and verify: every record checked

Every record against its field's own rule and against a partner field, in code; a second opinion from the model only where a check disagrees.

1 validate.py — 603 lines

The command

- **run_document()** sections, absence, checks, verification, score and route, then write
- **reroute_document()** recompute checks and routing without any model call
- **attach_sections()** the section each citation falls in
- **split_absence()** not found versus not stated in the document

2 validators.py — 5,598 lines

108 validators and 70 cross-checks

- **validate_record()** the field's own rule: dates in order, amounts positive, a pricing grid that steps in order
- **cross_check_record()** a partner field: lender rows adding up to the commitment, a maturity after the closing date
- **build_context()** the document, all its records and the filing dates
- **not-applicable reasons** why a check could not run: never a silent pass

3 verify.py — 483 lines

The second opinion

- **qualifies(), select()** records a check disagrees with, the most important fields first, up to 60
- **verify_record()** the model re-reads the passage, at low effort
- **attach_recovered_citation()** gives a record the passage the model cited

Tests: test_validators, test_partner_cross_checks, test_ca02_completeness, test_phase4_rulings, test_verify: 111

Stage 5, Score and route: what a reviewer sees first

The confidence score, the review decision and the record of how each value was obtained and what it cost.

1 confidence.py — 166 lines

The score

- **composite()** from the citation, the checks and the second opinion
- **band()** green, amber or red

2 tiers.py — 192 lines

Materiality

- **tier_for()** each field's review tier

3 route.py — 415 lines

The review decision

- **route_record()** to review or not, with every reason
- **parse_disposition()** the dictionary's rules, such as “auto-accept at 0.98 with a maths check”
- **in_tier_2_sample()** a stable sample of the lower tier

4 provenance.py — 333 lines

How each value was obtained

- **annotate()** the method, the model, the prompt and the passages sent, on every record

5 costing.py — 194 lines

What each field cost

- **apportion()** each call's cost shared across the records it produced
- **by_field()** cost per field

Tests: test_confidence, test_route, test_tiers, test_provenance, test_costing: 58

Stage 6, Outputs: what the client receives

Every deliverable built from the validated records by one command, with no model call.

1 common.py — 548 lines

Shared

- **load_bundle(), write_table()** loading and the house spreadsheet style

2 template.py — 325 lines

The filled template

- **build()** the client's field template, filled in

3 review.py — 718 lines

Review sheets

- **build_all()** one HTML review sheet per document

4 corrections.py — 257 lines

The golden set's sheet

- **build()** the corrections workbook reviewers fill in

5 json_output.py — 519 lines

JSON for systems

- **build_schema(), document_json()** one file per document, keyed by field name
- **validate_export()** checked against its schema

6 metrics.py — 562 lines

Metrics

- **collect(), write_markdown()** what the pipeline can see about its own run

7 vocabularies.py — 311 lines

Vocabularies

- **build()** the controlled lists, as a workbook

8 __main__.py — 49 lines

The command

- **build_all()** every deliverable in one run

Tests: test_outputs, test_json_output: 29

Around the pipeline: the record, the timeline, the tools

The record every stage reads and writes, the run timeline, and the tools that let people run the engine and see inside it.

1 contract.py — 132 lines

The record

- **Record** one field instance: value, quotes, checks, score, routing and the prompt and schema versions
- **Citation** a quote and its exact offsets in the text
- **CheckResult** pass, fail, not applicable or pending

2 trace.py — 82 lines

The run timeline

- **Tracer** one line per stage and per model call, with its time and cost, for every run

3 ui/server.py — 1,176 lines

The tool's server

- **Handler** records, marks, boxes, exports, the trace page and the batch endpoints; standard library only

4 jobs.py — 1,013 lines

The batch runner

- **Runner** filings through prepare, extract and validate, with progress, score, spend, cancel and exports

5 estimates.py — 462 lines

Cost and time

- **batch_estimate()** from the logs, with the budget headroom
- **remaining_seconds()** time left at the pace so far

6 fieldtrace.py — 358 lines

The field trace

- **trace_field()** one field walked through nine stages, for the trace page

7 ui/coverage.py — 166 lines

Golden-set coverage

- **main()** labels per field, and where

8 ui/merge.py — 351 lines

Reviewers' work back in

- **main()** folds returned annotation files into the repository

9 ui/share.py — 165 lines

Share the tool

- **main()** packages the tool and documents for another laptop

Tests: test_jobs, test_estimates, test_fieldtrace: 41; the record is used by nearly every test file